МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий

Допустить к защите
Заведующий кафедрой
канд. физ.-мат. наук, доц.
_____ В.В. Подколзин
(подпись)
_____ 2026г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА (БАКАЛАВРСКАЯ РАБОТА)

АВТОМАТИЗИРОВАННАЯ ГЕНЕРАЦИЯ РЕФЕРАТИВНЫХ ТЕКСТОВ НА ОСНОВЕ МОДЕЛЕЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Работу выполнил _____Н. С. Филипенко
(подпись)

Направление подготовки 02.03.03 Математическое обеспечение и администрирование информационных систем
(код, наименование)
Направленность (профиль) Технология программирования

Научный руководитель
канд. пед. наук, доц. _____Н. Ю. Добровольская

(подпись)

Нормоконтролер
канд. пед. наук, доц. _____ А. В. Харченко
(подпись)

Краснодар
2026

РЕФЕРАТ

Выпускная квалификационная работа содержит 51 с., 20 рис., 3 таблицы, 17 источников, 1 приложение.

АВТОМАТИЧЕСКОЕ РЕФЕРИРОВАНИЕ, ТРАНСФОРМЕРЫ, ОБРАБОТКА ЕСТЕСТВЕННОГО ЯЗЫКА, МАШИННОЕ ОБУЧЕНИЕ, ЭКСТРАКТИВНАЯ СУММАРИЗАЦИЯ, АБСТРАКТИВНАЯ СУММАРИЗАЦИЯ, ГЕНЕРАЦИЯ ТЕКСТОВ.

Цель работы – проектирование и реализация приложения автоматизированной генерации реферативных текстов.

В процессе работы изучены методы автоматической суммаризации текстов, проведен анализ существующих нейросетевых моделей и статистических методов и рассмотрены метрики оценки качества суммаризации.

В практической части спроектировано и разработано приложение, а также обучена модель для задачи абстрактивного реферирования русскоязычных научных текстов. Разработанное приложение обладает функционалом по вводу текста или загрузке файлов, выбору метода суммаризации и заданию параметров генерации, генерации реферата с помощью нейросетевых моделей архитектур BART и T5 и экстрактивных алгоритмов TextRank, LSA и LexRank, а также сохранению истории запросов для авторизованных пользователей.

Использованы средства разработки: язык программирования Python, фреймворк Django и Django Rest Framework, библиотеки pypdf, python-docx, hugging face transformers, pytorch, evaluate, sumy, nltk.

СОДЕРЖАНИЕ

Введение 3
1 Обзор современных исследований задачи автоматического реферирования 5
1.1 Современные тенденции автоматизированной генерации реферативных текстов 5
1.2 Основные этапы подготовки текстов 8
1.3 Обзор моделей и методов реферирования текстов 9
1.3.1 Экстрактивные методы 9
1.3.2 Абстрактивные методы 15
1.4 Критерии оценки качества реферативных текстов 21
2 Задача суммаризации научных статей 24
2.1 Требования к структуре рефератов научных статей в соответствии с ГОСТ 24
2.2 Некоторые программные решения для автоматического реферирования 26
3 Проектирование приложения для автоматизированной генерации реферативных текстов 29
4 Особенности реализации приложения 35
4.1 Использованные модели 35
4.2 Функционал приложения 36
5 Дообучение модели rut5 для решения задачи реферирования научных статей 40
6 Описание GPT-активностей 42
7 Рефлексивный анализ 45
Заключение 46
Список использованных источников 47
Приложение А Сравнение сгенерированных рефератов научных статей 50

ВВЕДЕНИЕ

Потребность в реферировании значительно возросла с экспоненциальным ростом объема текстовых данных, поэтому автоматическая суммаризация текста – важная и актуальная задача в сфере обработке естественного языка, направленная на создание краткого и содержательного представления более длинного исходного текста. На основе текстов сгенерированных рефератов реферирование можно разделить на два типа: экстрактивное и абстрактное. В экстрактивном реферировании сгенерированный реферат состоит из слов и предложений, непосредственно извлеченных из исходного текста, тогда как в абстрактном реферировании краткий реферат содержит ключевые концепции исходного текста в новом сгенерированном тексте. Сгенерированный реферат потенциально включает фразы и предложения, отсутствующие в исходном тексте.

Первая глава выпускной квалификационной работы посвящена обзору современных исследований, методов и критериев оценки качества в области автоматического реферирования.

Во второй главе приводятся требования к структуре рефератов научных статей в соответствии с ГОСТ, а также рассматриваются программные решения задачи автоматического реферирования научных текстов.

Третья глава содержит построение различных диаграмм, определение задач и требований к ним.

В четверной главе описываются особенности реализации приложения для автоматизированной генерации реферативных текстов.

Пятая глава посвящена дообучению модели архитектуры T5 для решения задачи реферирования научных статей, а также сравнение и оценка рефератов, полученных различными методами.

Шестая глава содержит документирование GPT-активностей, произведенных в процессе работы. Глава включает матрицу промптов, содержащую промпт, его цель, результат и проведенный фактчекинг.

В седьмой главе проанализирована успешность кейсов и сформулированы рекомендации для дальнейшей работы.

1 Обзор современных исследований задачи автоматического реферирования

1.1 Современные тенденции автоматизированной генерации реферативных текстов

На данный момент существует достаточно большое количество отечественных и зарубежных статей на тему автоматического реферирования текстов, которые говорят о важности решаемой задачи, когда объемы и количество текстов растут и это приводит к трудностям в работе специалистов [1-6].

В своем исследовании С. Г. Сорокина представляет глубокий анализ различных подходов к проблеме, говоря о том, что автореферирование становится необходимым инструментом для науки, образования, бизнеса, медицины и множества других областей, так как позволяет быстрее понимать суть и ключевые мысли текстов. Также она выделяет различные классификации рефератов и методы их построения. Так, автор говорит о недостатках подходов: экстрактивный текст может выглядеть как набор предложений и быть не связан, тогда как абстрактивный текст хоть и похож на написанный человеком, но может содержать в себе искажения. При проведении сравнений при помощи традиционных метрик, таких как ROUGE, BLEU, METEOR автор говорит о превосходстве гибридных методов [1].

Работа Ю.В. Медяник и Л.А. Сабировой посвящена анализу экстрактивных методов автосуммаризации для работы с научными статьями. Так, авторы на основе структуры научный работы: аннотация, введение, обзор литературы, методология, результаты, обсуждение и выводы, делают вывод о необходимости использования основной информации из разделов с наиболее высокой практической значимостью текста, то есть из разделов, где обсуждаются используемые методы и общие результаты статьи. Авторы проводят обзор извлекающих методов и после их анализа останавливаются на использовании статистического TF-IDF для быстрой фильтрации важных

терминов и графового TextRank для общей связной структуры текста. В результатах работы авторы говорят о необходимости проведения экспериментов на основе полученных особенностей [2].

Напротив, в своем исследовании Д. В. Мельничук и А. В. Носкина проводят сравнение различных NLP-моделей применимо к задаче автореферирования научных текстов на русском языке. Так, они исследуют предобученные модели с архитектурой трансформер с генерирующим подходом к проблеме. В работе сравниваются 3 модели, для которых ставится задача генерация аннотация на основе текста статьи и дальнейшем сравнении с аннотацией автора. Модели mBART, T5, GPT-3 авторы работы предобучают на корпусе Gazeta Ильи Гусева, составленном из заголовком новостных статей с их содержанием, а для непосредственного fine-tuning собран датасет из 825 статей с сайта КиберЛенинка. По результатам исследования по большинству метрик лидирует T5, но в общем авторы выделяют слабости моделей в терминологии, что они связывают с направлением корпуса для предобучения [3].

В этом же направлении исследования можно выделить статью под авторством А.Е. Дагаева и Д.И. Попова, в которой они также проводят анализ эффективности современных моделей для генерации краткого содержания текстов на русском языке GigaChat, YaGPT2, ChatGPT-3.5, ChatGPT-4. Авторы работы проводят сравнение с предобучением на таких корпусах, как Gazeta, XL-Sum, WikiLingua. В конце авторы приводят сравнение с помощью стандартных метрик, а также вводят свою, являющуюся взвешенной суммой ранее упомянутых метрик с учетом степени сжатия текста. Авторы отмечают превосходство GigaChat для текстов на русском языке, но отмечают такие же проблемы, как и авторы статьи выше: разная предметная область обучающих корпусов и непосредственных текстов выливается в не очень глубокий по терминам реферат. Также, как и в предыдущей статье авторы отмечают отсутствие экспертной оценки реферативных текстов, что также может нести проблему [4].

В работе же К.В. Ребенка проводится исследование различных архитектур и подходов по отношению к проблеме и их применимость к задаче на русском языке. Так, автор исследует рекуррентные нейронные сети и трансформеры, которые подходят для обработки длинных последовательностей, а также TextRank. В результате исследования автор делает вывод о том, что трансформер превосходит сети с долгой кратковременной памятью по метрике ROUGE-1, из-за того, что трансформер использует механизм внимания. При этом проводится экспертная оценка 50 текстов и выделяется такие известные проблемы, как склонность к галлюцинациям у моделей с архитектурой BERT, потеря точности и терминов по сравнению с оригинальным текстом у GPT, отсутствие общей связности у TextRank. Автор, говоря о перспективах, отмечает использование специфического по области и большего по объему корпуса, развитие гибридных методов и использование мультимодальных подходов для извлечения информации не только из текста, но и из изображений, таблиц и графиков [5].

Проблема отсутствия обучающих корпусов прослеживаются не только на русском, но и на многих других языках. Так, в исследовании Mehmet Samet Duran и Tevfik Aytekin этому аспекту выделена ключевая роль и разработана система с учетом лингвистических особенностей турецкого языка. Модель авторов предобучена на mBART с учетом метрики, разработанной специально для турецкого языка, и собранного датасета с выделением уровня подготовленности читателя. Такая архитектура превосходит обычные архитектуры на основе трансформеров на 15 процентов в способности генерировать реферат с необходимым уровнем подготовки. Несмотря на успехи, модель не оценивается экспертами, поэтому сложно понять реальную эффективность модели, из-за вычисления отклонения от разработанной метрики обучение вычислительно затратно, а в целом обучение усложнено явлением агглютинации в турецком языке. Но авторы отмечают ценность подхода в других низкоресурных языках [7].

Таким образом, исследования показывают, что переход к абстрактивным сложным методам на основе трансформеров обоснован и они показывают лучшие результаты, чем извлекающие методы. Тем не менее необходимо выбирать подходы, методы и метрики, подходящие конкретному языку, задаче или области. Общей проблемой является отсутствие датасетов и обучающих корпусов для конкретной проблемы, вычислительная сложность и необходимость проверки экспертами готового реферата, так как метрика не может установить, насколько текст удобен и читабелен для человека [1-5,7].

1.2 Основные этапы подготовки текстов

С.Г. Сорокина в своем исследовании проводит подробный обзор сервисов, методов и приемов автоматического реферирования [1]. Кроме того, говорит о том, что для продуктивного процесса как реферирования, так и других задач обработки естественного языка, таких как машинный перевод, анализ тональности и т.д., необходимо провести предобработку исходных данных, которая может состоять из таких этапов, как:

– очистка текста от несущественных элементов, таких как нумерация, излишняя табуляция, изображения, знаки препинания;

– перевод всех букв в единый регистр для исключения ситуации принятия их за разные слова;

– обнаружение и удаление стоп-слов (имеющих частое появление, но не несущих смысла для всего текста в целом);

– процесс токенизации или разбиения текста на отдельные лексические единицы (слова, словосочетания и предложения);

– процесс лемматизации или приведение слова к его нормальной словоформе (например, «ложками» переводит в «ложки»);

– процесс стеминга или приведение слова к его корневой форме за счет отбрасывания морфем (например, «строители» переводит в «строит»);

– добавление структуры текста в качестве метатегов или заголовков для понимания алгоритмом.

1.3 Обзор моделей и методов реферирования текстов

В зависимости от поставленной задачи, применяемых методов, способов обработки, предназначения реферата выделяют основные виды реферирования: экстрактивное и абстрактивное.

Экстрактивное реферирование характеризуется формированием нового текста из фрагментов (фраз, предложений, абзацев) исходного документа. Итоговый реферат не содержит других элементов, кроме тех, которые встречались в исходном тексте.

Так, в работе [2] выделяются и анализируются наиболее широко используемые методы экстрактивного, среди которых выделяют статистические (Метод Луна, TF-IDF, LSA) и графовые (TextRank, LexRank), которые не нуждаются в эталонных рефератах и не привязаны к определенной предметной области, что делает их быстрее и проще, и методы на основе машинного обучения для обработки последовательностей, использующие рекуррентные нейронные сети или модели, использующие архитектуру Transformer. Рассмотрим подробнее каждый из них далее.

1.3.1 Экстрактивные методы

Рассмотрим методы, которые вычисляют последовательно важность предложений, являющиеся чисто статистическими.

Первым из методов реферирования считается метод Луна, появившийся в 1958 году и состоящий в следующей последовательности шагов [11]

Проводим стемминг или лемматизацию слов, чтобы разные словоформы одного слова могли считаться одним и тем же. Затем считаем частоты слов, формируем их список по убыванию. Убираем стоп-слова: часто

встречающиеся слова, у которых нет самостоятельной смысловой нагрузки, такие как предлоги, союзы, частицы. Убираем слишком редкие слова, например такие, которые встречаются только 1 раз, либо убираем какой-то нижний перцентиль слов по частоте. Все оставшиеся слова считаем значимыми.

Далее предложение делим на промежутки, которые начинаются и заканчиваются значимыми словами. В промежутке могут встретиться и незначимые слова, но не более 4 подряд. Важность промежутка вычисляется, как квадрат количества значимых слов в промежутке, делённый на размер промежутка. Значимость предложения представляет собой максимум из важностей промежутков. Берём в качестве реферата предложения со значимостью выше определённого установленного порога.

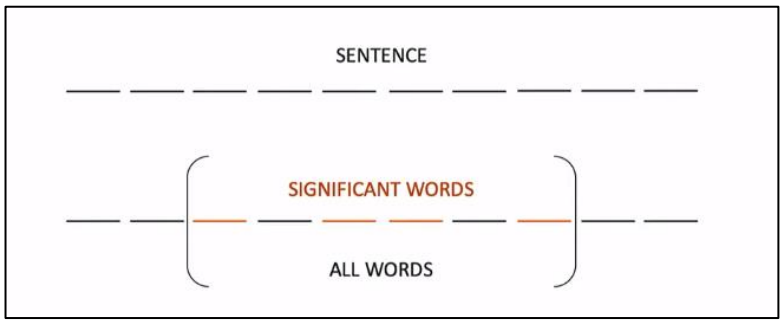


Рисунок 1 – Графическое представление работы метода

Существует множество методов, не только для задачи реферирования, в основе которых лежит статистическая мера TF-IDF (Term Frequency – Inverse Document Frequency), которая имеет предназначение для вычисления значимости слов в конкретном контексте путем учета их частотности по сравнению с их встречаемостью во всем корпусе.

TF – это частота встречаемости слова в документе, которая показывает значимость слова в тексте, вычисляемая по формуле (1).

$$TF(t,d) = \frac{n_t}{\sum_k n_k} \quad (1)$$

где

n_t – количество употреблений терма t в документе;

$\sum_k n_k$ – общее количество слов в документе.

IDF отвечает за уникальность слова среди всех анализируемых документов, вычисляется по формуле (2).

$$IDF(t, D) = log \frac{|D|}{|\{d_i \in D \mid t \in d_i\}|} \quad (2)$$

где

$|D|$ – количество документов;

$|\{d_i \in D \mid t \in d_i\}|$ – общее количество документов, содержащих терм t.

Если слово часто встречается в конкретном документе, его вес будет большим относительно других единиц в этом тексте. Напротив, вес слова будет небольшим, если оно встречается во многих документах. Соответственно, словам, которые встречаются в большом количестве документов, присваивается низкое значение IDF, так как они не имеют высокой значимости. При суммаризации методом на основе TF-IDF важность предложений для дальнейшего составления резюме рассчитывается как сумма важностей отдельных слов внутри каждого предложения. Чем больше получившееся число, тем значимее предложение. Конечный текст формируется из предложений с наивысшими баллами.

Графовые методы, такие как TextRank, представляют текст как граф, где вершины являются предложениями или словами, а ребра – связями между ними (например, совпадение слов или семантическая близость).

Таким образом общий порядок алгоритма имеет следующий вид.

– определение лексических единиц (слов или предложений) и внесение их в граф как вершины;

– определение связи по формуле (3) между этими вершинами (посредством непосредственной близости в тексте или семантической близости) и построение ребер графа:

$$sim_{ij} = \frac{\left|\{w|(w \in S_i) \cap (w \in S_j)\}\right|}{\log(|S_i|) + \log(|S_j|)} \quad (3)$$

где

w – отдельное слово, входящее в набор слов предложения;

S_i – набор слов i-го предложения;

S_j – набор слов j-го предложения.

– посредством алгоритма ранжирования строятся веса (pagerank) до тех пор, пока изменения не станут незначительными, по формуле (4):

$$P(S_i) = \frac{(1-d)}{|S|} + d(\sum_{S_j} \frac{sim_{ij}}{\sum_{S_k} sim_{kj}} P(S_j)) \quad (4)$$

где

$P(S_i)$ – PageRank i-го предложения;

sim_{ij} – схожесть S_i и S_j;

$|S|$ – мощность множества всех предложений;

d – коэффициент демпфирования.

– далее вершины сортируются в порядке убывания и предложения с наивысшими оценками выбираются для реферата.

Улучшением алгоритма метода TextRank является LexRank, который получает выигрыш за счет использования модификации TF-IDF и вычисления меры схожести на ее основе [11]. Это позволяет меньше учитывать часто встречающиеся слова, которые скорее относятся к общим, несущих меньше

информации, чем термины, а на сами специальные термины обращать больше внимания [2].

Схожесть предложений в этом случае определяется по формулам (5), (6) и (7) ниже.

$$tf_w^i = \frac{|\{w_k | (w_k \in S_i) \cap (w_k = w)\}|}{|S_i|} \tag{5}$$

$$idf_w = log \frac{|S|}{|\{S_i \mid w \in S_i\}|} \tag{6}$$

$$sim_{ij} = \frac{\sum_{w \in (S_i \cup S_j)} tf_w^i tf_w^j (idf_w{}^2)}{\sqrt{\sum_{w \in S_i} (tf_w^i idf_w)^2} \sqrt{\sum_{w \in S_j} (tf_w^j idf_w)^2}} \tag{7}$$

Пример итоговой матрицы схожести представлен на рисунке 2.

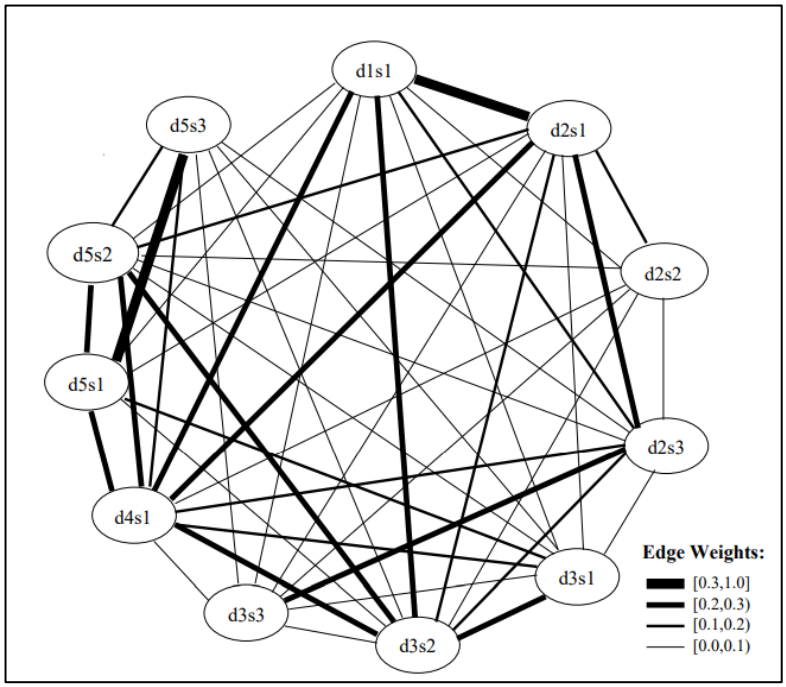


Рисунок 2 – Граф схожести

Во всех этих методах мы не учитываем темы, из которых состоит исходный текст. Для решения этой проблемы можно использовать метод латентно-семантического анализа, который основывается на разложении исходной матрицы инцидентности из слов и предложений на три отдельные матрицы по формуле (8).

$$A = USV^T \tag{8}$$

где

S – диагональная матрица с неотрицательными элементами, у которой элементы, лежащие на диагонали – это сингулярные числа;

U – матрица, столбцы которой представляют собой левые сингулярные векторы;

V – матрица, столбцы которой представляют собой прямые сингулярные векторы.

Кроме того, i-тый столбец третьей матрицы соответствует i-тому предложению, а его компоненты – величине важности одной из тематик в этом предложении, а элементы средней матрицы – значимости этой тематики. То есть k-тый сингулярный вектор матрицы V^T соответствует теме k и при взятии максимального значения из этого столбца мы получим лучшее предложение по этой теме.

В итоге мы оставляем установленное количество самых важных тематик и на их основе берем самые подходящие предложения по этим темам [11].

Несмотря на всю привлекательность и легкость методов без обучающих примеров, более точное представление краткого содержания дают методы на основе нейронных сетей, которые мы рассмотрим далее.

Одним из первых является метод SummaRuNNer, представляющий собой по архитектуре двунаправленную рекуррентную сеть на основе GRU, первое направление которой проходится по отдельным словам, а второй – по всем предложениям. Общая архитектура показана на рисунке 3 [12].

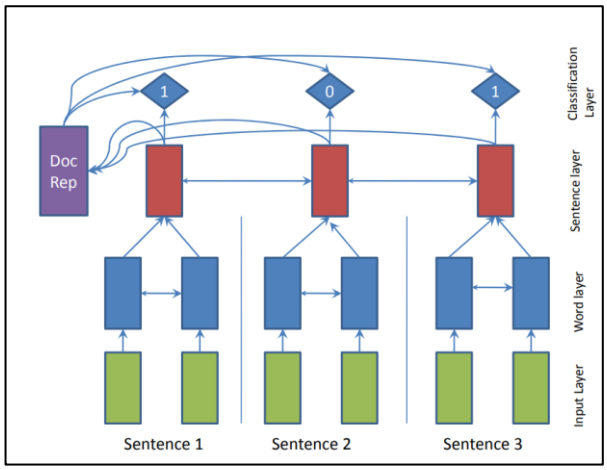


Рисунок 3 – Архитектура SummaRuNNer

Алгоритм метода в целом представляет собой бинарную классификацию на вопрос включения предложения в реферат. Определяется это решение схожестью предложения на исходный текст на основании какой-либо оценки или метрики, например ROUGE, BLEU или METEOR.

1.3.2 Абстрактивные методы

В противовес рассмотренным выше экстрактивным методам можно также привести наиболее актуальные в данный момент времени абстрактивные или генерирующие методы решения задачи составления реферативных текстов. При этом подходе текст на выходе получается похож на человеческий: выглядит связно и синтаксически последовательно. И методы, с помощью которых он составляется тоже похожи на рассуждения и методы человека при написании краткого содержания. Так нейронная сеть заменяет похожие понятия на синонимы, упрощает предложения и термины, обобщает абзацы предложениями, которые рассказывают о чем будет абзац, из начала и фразами в конце абзаца, подводящие некий промежуточный итог [13].

Так, одним из первых значимых подходов к задаче абстрактивного реферирования стал Sequence-to-sequence (Seq2seq). Его суть заключается в переводе одной последовательности токенов в другую и изначально Seq2seq

был придуман для перевода. Применимо к задаче суммаризации – перевод токенов исходного текста в выходные токена реферата. Стандартным в этом подходе является разделение на кодировщик и декодировщик. Так кодировщик читает входной текст и создает его представление в виде токенов x, а декодировщик на их основании генерирует выходной текст y. Пример представлен на рисунке 4 [14].

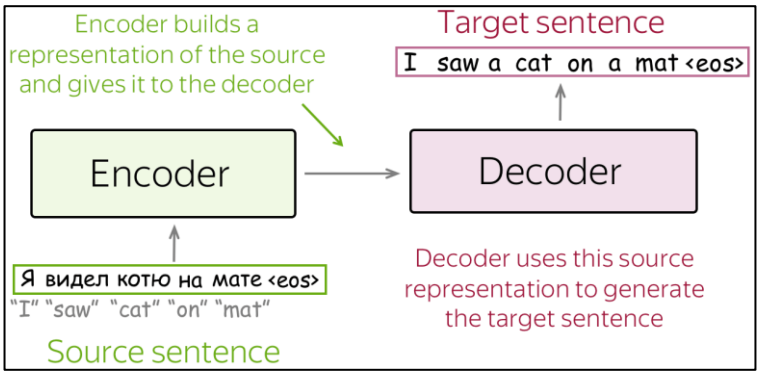


Рисунок 4 – Пример работы Seq2seq

Работает декодировщик, как предсказатель следующего токена на основе предыдущих сгенерированных слов y_t и вектора из кодировщика x, вычисляя вероятности появления по формуле (9).

$$P(y_1, y_1, \dots, y_n|x) = \prod_{t=1}^{n} p(y_t|y_{<t}, x) \tag{9}$$

Наиболее простая модель на основе кодировщика и декодировщика состоит из двух рекурентных нейронных сетей на основе LSTM. В этой модели последнее скрытое состояние кодировщика становилось первым скрытым состоянием декодировщика. При обучении модели при этом применялась функция потерь в виде кросс-энтропии нужного распределения вероятностей p* и полученными вероятностями p, рассчитываемая по формуле (10).

$$Loss(p^*, p) = -\log(p_{y_t}) \tag{10}$$

Проблема такого подхода заключается в том, что кодировщик делает только одно представление текста, поэтому модель может забыть что-то важное. Это становится проблемой и для декодировщика, которому могут понадобиться разные части текста в разные моменты времени.

Для решения этой задачи был введен новый механизм, получивший название механизм внимания. На каждом шаге нейронной сети с использованием внимания декодировщик решает какая часть наиболее важная, а кодировщику больше не нужно сжимать целый текст в один вектор, теперь он строит представления для всех токенов. Таким образом, работу декодировщика можно описать в следующих шагах:

– получает состояние декодировщика и все состояния кодировщика;

– вычисляет оценку внимания на основании их релевантности для нынешнего состояния декодировщика;

– вычисляет веса внимания с помощью функции softmax;

– считает взвешенную сумму состояний кодировщика [14].

Пример работы такого подхода показан на рисунке 5.

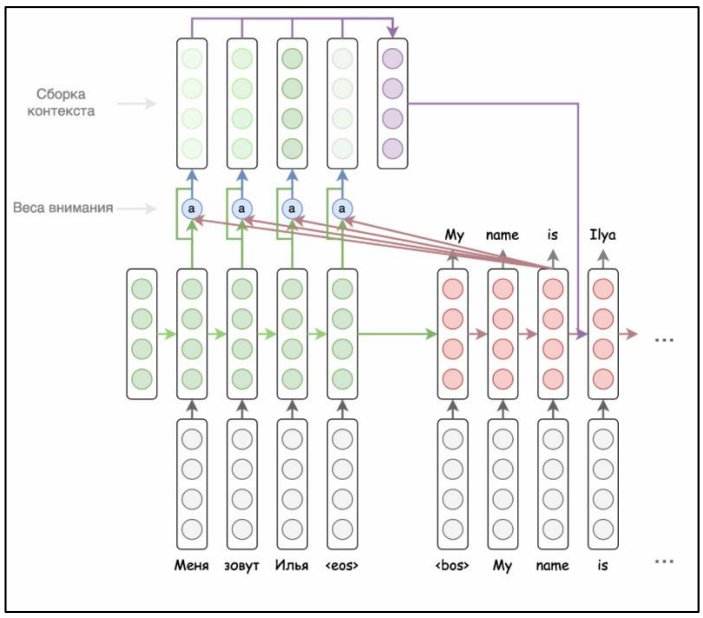


Рисунок 5 – Механизм внимания

В настоящее время стандартом для задач превращения последовательности в другую последовательность является подход, основанный только на механизме внимания, под названием трансформер. Помимо более высокого качества, модель на его основе обучается на порядки быстрее.

Механизмы работы кодировщика и декодировщика также отличаются от предыдущих примеров. Так, кодировщик повторяет некоторое число раз просмотр всех токенов друг на друга и обновление их представлений, а декодировщик это же число раз смотрит на предыдущие получившиеся и исходные токены и обновляет их представления.

Одним из важных компонентов модели также является самовнимание. Заключается оно в просматривании в каждом из состояний других состояний, относящихся к этому же шагу. То есть каждый токен, смотрим на другие токены в предложении, собирает данные и обновляется свои представления. При этом токен получается три представления: запрос q, ключ k и значение v. Далее внимание вычисляется по формуле (11).

$$Attention(q, k, v) = softmax\left(\frac{qk^T}{\sqrt{d^k}}\right)v \qquad (11)$$

где

d – размерность векторов.

В случае же самовнимания декодировщика маскируются будущие токены за следующим, что позволяет декодировщику наблюдать только за предыдущими.

Также важен механизм многоголового внимания, позволяющий понимать отношение отдельного слова в других частях предложения. Считается такое внимание как соединение разных голов, головы же в свою очередь вычисляются как внимание отдельных слов. Итоговая архитектура представлена на рисунке 6.

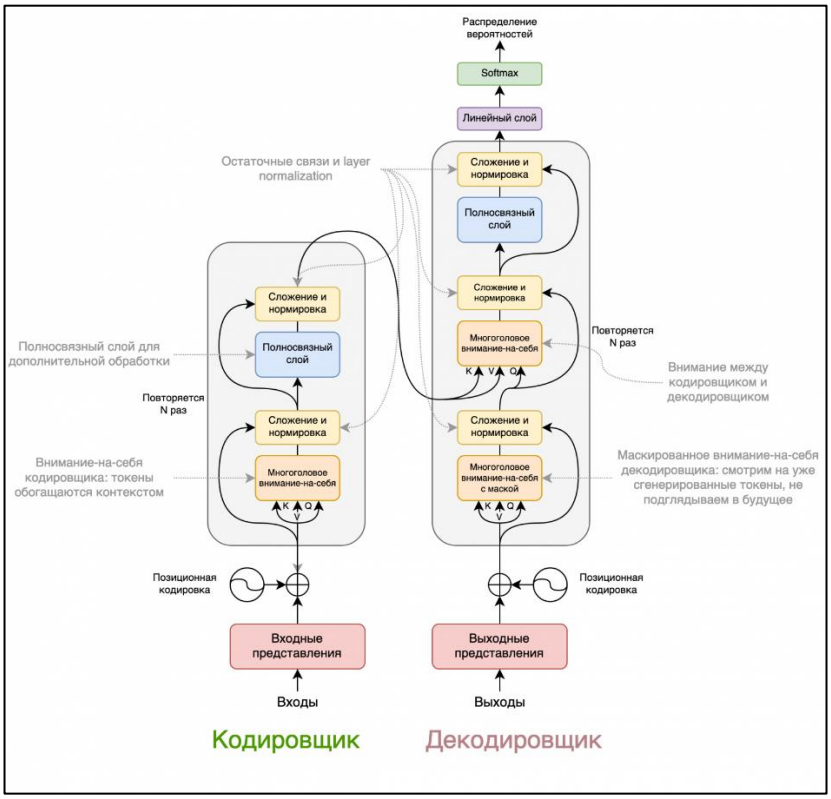


Рисунок 6 – Трансформер

Одной из наиболее популярной архитектурой для решения задач обработки естественного языка является BERT (Bidirectional Encoder Representations from Transformers), ее отличием от других является понимание текста как слева, так и справа от слова, что делает модель лучше понимающей связи между словами. Тренируется модель на двух задачах: восстановлении замаскированных слов, так, например, модель должна восстановить текст, где пропущены 15% слов текста, и предсказывания порядка двух предложений для понимания логической связи или ее отсутствии [14].

Существует множество специальных моделей на основе архитектуры трансформер для решения задачи абстрактивного автоматического реферирования. Рассмотрим их далее.

Модель BertSumAbs на основе Bert создает новый текст в sequence-to-sequence подходе. В качестве кодировщика используется трансформер из 6 слоев [13].

Семейство моделей GPT основано на предобучении предсказания следующего идущего слова в тексте слева направо. Конкретно для задачи составления реферата текста обучающий реферат разделяются с помощью некоторого символа, а затем модель пробует создать реферат на основе текста, который она встретила ранее. Для русского языка компанией Сбербанк было создано семейство моделей ruGPT.

Также следует отметить модель на основе sequence-to-sequence трансформера BART, созданную компанией Facebook, которая обучалась на восстановлении оригинала испорченного зашумленного замаскированного текста. В тексте маскируются или удаляются отдельные токены, случайно сортируются предложения, а некоторые группы слов маскируются как один токен. Ее особенностью является использование как кодировщика, так и декодировщика. Различие показаны на рисунке 7.

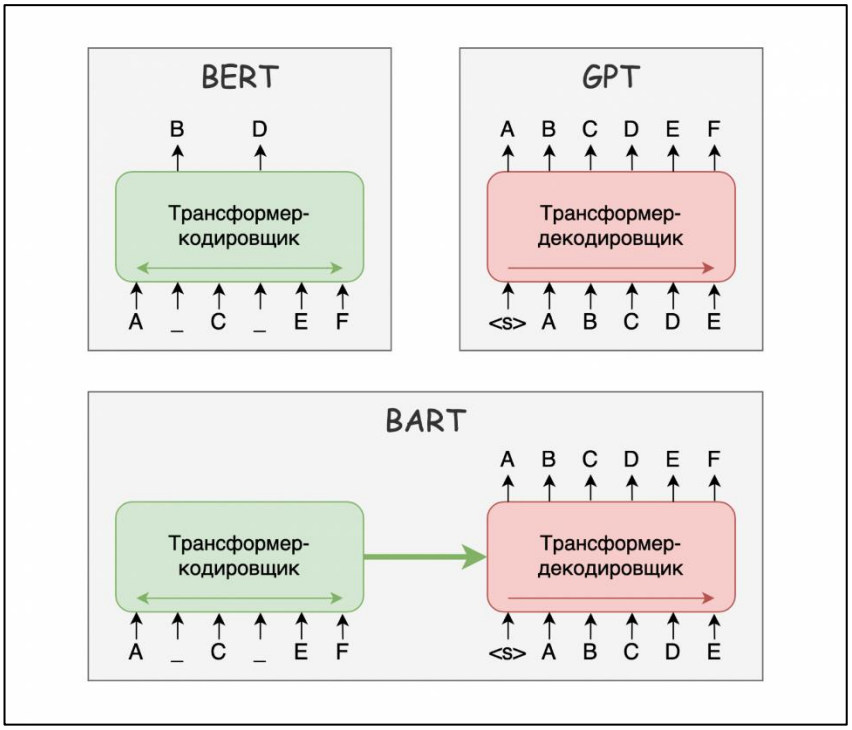


Рисунок 7 – Различия моделей BERT, GPT и BART

Компанией Google также представлен своя архитектура на основе sequence-to-sequence трансформер под названием T5(Text-to-Text Transfer

Transformer). Модель при обучении решала проблема заполнения пропусков целых кусков предложения.

Существует также модель PEGASUS, созданная специально для задачи генеративного реферирования. Ее же задачей при обучении являлась создание пропущенных предложений. При этом каким-либо образом выбирались предложения для составления экстрактивного реферата, затем они удалялись, а модели предлагалось их сгенерировать [13].

1.4 Критерии оценки качества реферативных текстов

Для оценки различных параметров полученного реферата используется множество метрик, которые дают некоторую оценку тексту на основе исходного. Это может быть качество, читаемость, последовательность изложения, общая связность и т.д. В настоящее время такие количественные меры оценки можно разделить на традиционные (ROUGE, BLEU, METEOR) и нейросетевые (BERTScore, BLEURT). Рассмотрим конкретные примеры далее [1].

Самой используемой является метрика ROUGE и ее разновидности.

Она была создана как раз для оценки качества реферата. Так, ROUGE-1 считает количество совпадения отдельных слов(униграм) в реферате и оригинальном текста, ROUGE-2 - количество комбинаций из стоящих вместе двух слов(би-грам), ROUGE-3 - для трех соответственно и т.д. Также выделяется ROUGE-L, которая оценивает схожесть на основе самой длинной последовательности слов. Представим ниже формулы (12), (13) и (14) для расчетов на примере ROUGE-2.

$$ROUGE_{2_{recall}} = \frac{\text{Число одинаковых комбинаций из 2х слов}}{\text{Их число в исходном тексте}} \qquad (12)$$

$$ROUGE_{2_{presicion}} = \frac{\text{Число одинаковых комбинаций из 2х слов}}{\text{Их число в реферате}} \quad (13)$$

$$ROUGE_{2_{F1-score}} = 2\frac{precision * recall}{precision + recall} \quad (14)$$

Далее стоит рассмотреть BLEU (BiLingual Evaluation Understudy). Она считает количество совпадений для отдельных слов и словосочетаний. Потом это число делится на общее число слов и словосочетаний в получившемся переводе - получается precision. К итоговому precision применяется корректировка - штраф за краткость (brevity penalty), чтобы избежать слишком высоких оценок BLEU для кратких и неполных переводов. Приведем формулу (15) для словосочетаний длины до четырех слов.

$$BLEU = BP * \exp\left(\frac{1}{4}\sum_{i=1}^{4} \ln\left(p_i\right)\right) \quad (15)$$

где

BP – штраф за краткость, равный 1, если длина получившегося реферата больше эталонного, иначе – $\exp\left(1 - \frac{\text{длина эталонного реферата}}{\text{длина сгенерированного реферата}}\right)$;

p_i – длина n-грамы из i слов.

Проблемой данных оценок состоит в необходимости полного совпадения слов, что в случае генерирующего реферирования или машинного перевода нечасто, а порой и невозможно.

METEOR (Metric for Evaluation of Translation with Explicit ORdering) учитывает однокоренные слова и синонимы. Рассчитывается с учетом наказания за закучкованность по формуле (16).

$$METEOR = 10\frac{precision * recall}{precision + 9 * recall}\left(1 - w\left(\frac{ch}{m}\right)^3\right) \quad (16)$$

где

ch – число последовательных совпадений;

w – штрафной коэффициент;

m – число сопоставленных вместе униграм [6].

На основании BERT также создана метрика BERTScore, которая способна определять глубокие связи с помощью векторных представлений слов. После этого определяется их схожесть на основании косинуса угла между ними. Аналогично прошлым методам строится precision, recall и F1-Score.

Так, выбирать метрику стоит в зависимости от представленной задачи, например, для простой оценки точности ROUGE и BLEU, а для семантической близости следует использовать METEOR и BERTScore [8].

2 Задача суммаризации научных статей

2.1 Требования к структуре рефератов научных статей в соответствии с ГОСТ

Согласно ГОСТУ Р7.0.7–2021 статья в сборнике или научном журнале должна содержать следующие элементы издательского оформления:

- сведения об издании, в котором опубликована статья;
- название рубрики или раздела издания;
- тип статьи;
- индекс Универсальной десятичной классификации (УДК);
- цифровой идентификатор объекта (Digital Object Identifier – DOI);
- заглавие статьи;
- подзаголовочные данные статьи;
- сведения об авторе (авторах);
- аннотация (резюме);
- ключевые слова (словосочетания);
- благодарности;
- знак охраны авторского права;
- перечень затекстовых библиографических ссылок;
- сведения о продолжении или окончании статьи;
- приложение (приложения);
- примечания;
- дата поступления рукописи в редакцию издания, дата одобрения после рецензирования, дата принятия статьи к опубликованию [9].

Здесь стоит сосредоточиться на элементе аннотация или резюме, что является синонимом реферата. Так, необходимо рассмотреть предлагаемые к нему требования, которые подробно описаны в ГОСТ Р 7.0.99-2018 [10].

Под рефератом авторы подразумевают краткое точное изложение содержания первичного документа в текстовой форме, включающее основные

фактические сведения и выводы, без дополнительной интерпретации или критических замечаний автора реферата.

При этом приводится несколько классификаций по разным аспектам самого резюме. Так, например, по форме изложения выделяют:

– информативный реферат: реферат, отражающий в обобщенном виде все основные положения первичного документа;

– индикативный реферат: краткий реферат, отражающий основные темы и вид первичного документа;

– информативно-индикативный реферат: комбинированный реферат, отражающий в сокращенном виде основные положения и аспекты первичного документа.

Таким образом, авторы указывают, что информативный реферат должен иметь цель работы, методологию проведения работы, а индикативный - вид, форму документа. Оба вида обязаны содержать предмет, тему работы, результата, их область применения и выводы. Рекомендовано использовать информативный для основных фактов работы и для документов, которые могут описывать результаты экспериментов, а индикативный – при анализе обзоров, которые можно по содержанию причислить ко многим предметным областям. Рекомендуемый средний объем текста реферата – 850 печатных знаков.

А по составителю краткого содержания научной работы:

– авторский реферат (автореферат): реферат, составленный автором первичного документа;

– неавторский реферат: реферат, составленный референтом;

– машинный (автоматический) реферат: реферат, составленный с помощью компьютерной программы;

– аннотация: краткая характеристика первичного документа с точки зрения его назначения, содержания, вида, формы и других особенностей.

Аннотация должна в себя включать при этом сведения о содержании работы, его авторах, типе документа, основной теме, проблемах, объекте, цели работы и ее результатах, новизне и достоинствах документа, его научном и практическом значении для целевой аудитории.

Авторы также приводят свою точку зрения по методологии составления реферата, давая конкретные шаги и четко разделяя этапы проведения анализа работы.

Текст самого краткого содержания не должен субъективную оценку автора по содержанию работы, а отражать только исходную информацию, при этом сам текст должен быть лаконичным и не содержать повторений. Не стоит также обогащать реферат сложными грамматическими конструкциями, а ограничиваться научными и техническими терминами и фразами, которые свойственны конкретной предметной области. Рекомендуется также включать в состав ключевые или наиболее важные слова, которые часто встречаются в основном тексте научной работы [10].

2.2 Некоторые программные решения для автоматического реферирования

Рассмотрим примеры сервисов, осуществляющих составление реферата.

Для первого примера возьмем онлайн-сервис QuillBot [16], который предоставляет широкий функционал по работе с текстом, кроме реферирования который может улучшать и перефразировать текст, исправлять ошибки. Пример работы сервиса представлен на рисунке 8.

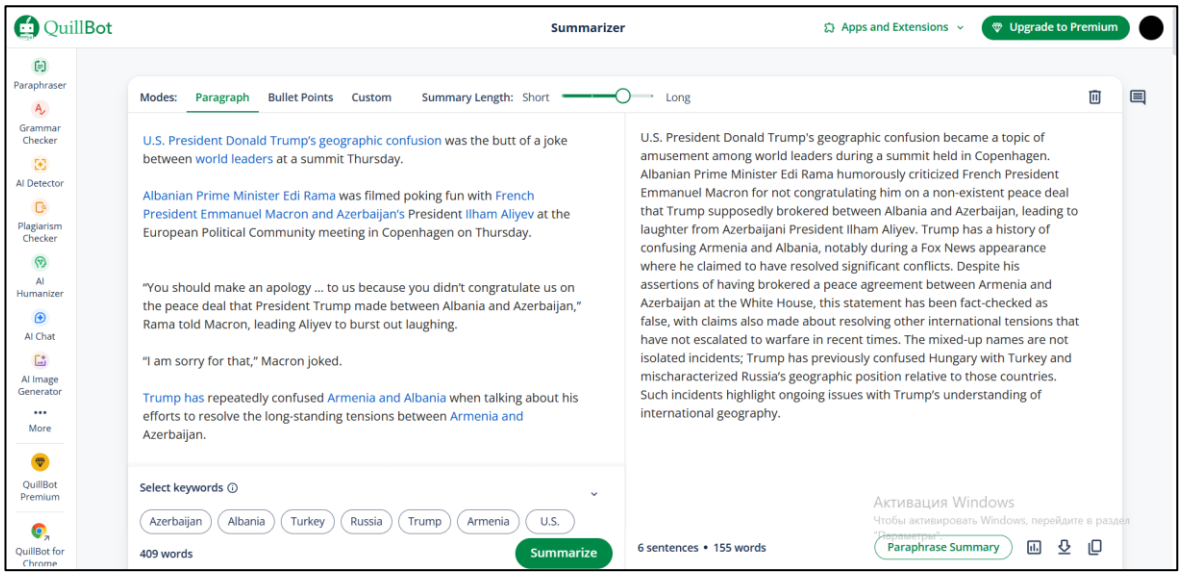


Рисунок 8 – Пример работы сервиса

Сервис предлагает три режима: реферата, ключевых фактов и настраиваемый. При этом можно выбрать длину реферата и выбрать обязательные для включения ключевые слов.

В рассматриваемом сервисе можно вставить текст, загрузить файл или вставить ссылку для извлечения текста. Получившийся реферат можно редактировать и экспортировать в удобном формате.

Вторым примером рассмотрим еще один онлайн-сервис SciSummary (рисунок 9) [17].

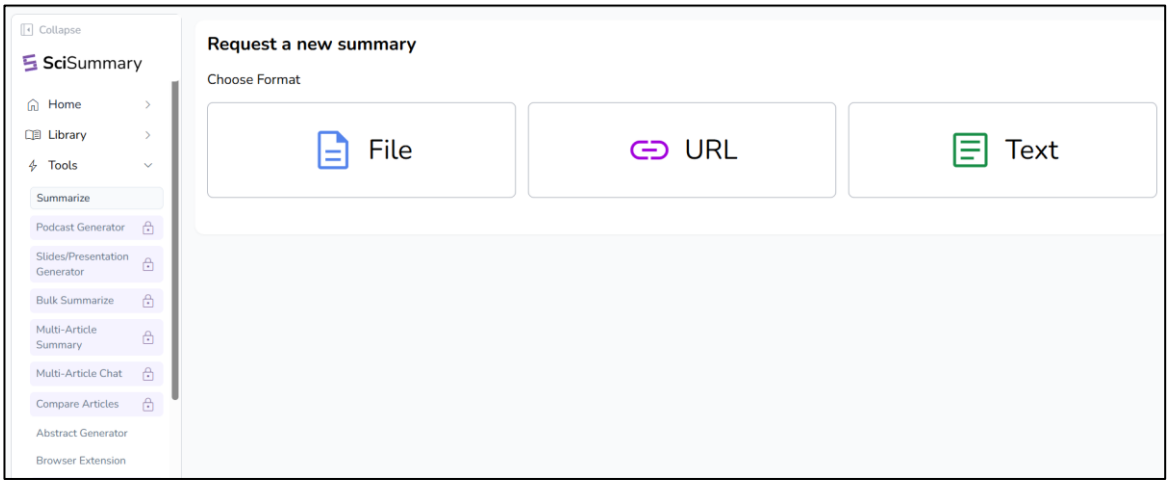


Рисунок 9 – Пример сервиса

Он специально предназначен для научного сообщества и предлагает широкий функционал для работы с научными работами можно производить их поиск как среди всех статей, так и среди вашей личной загруженной базой статей. Для составления реферата можно использовать файлы любых форматов, вставлять текст непосредственно или устанавливать ссылку на ресурс.

Также для такого же рода задач предназначается и сервис Scholarcy [15]. Он предоставляет возможность создания 3 бесплатных реферата научной работы при этом выделяя, как общий реферат статьи, так и резюме каждого из его подразделов. Пример работы представлен на рисунке 10 [1].

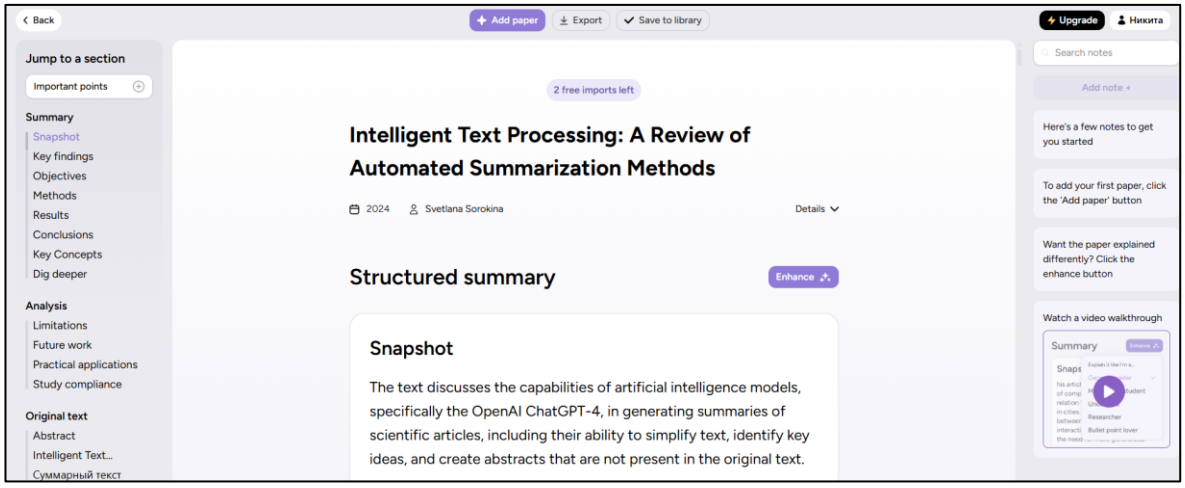


Рисунок 10 – Пример работы

3 Проектирование приложения для автоматизированной генерации реферативных текстов

Опишем основные этапы проектирования приложения автоматизированной генерации рефератов. Входная и выходная информация, структурная схема типа IDEF0 представлена на рисунке 11.

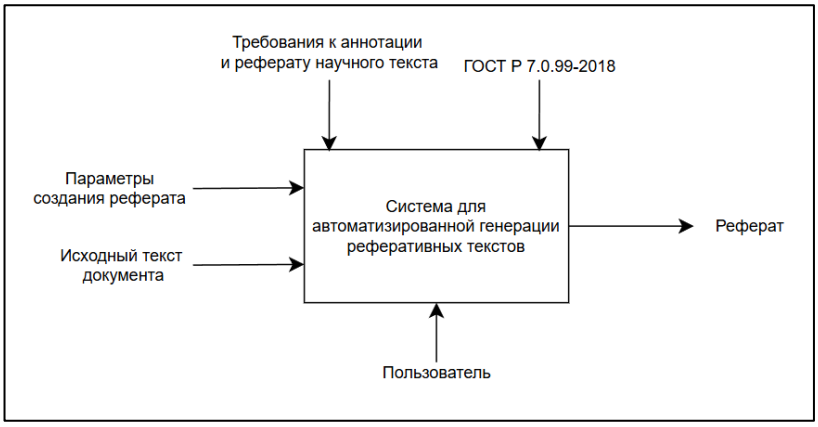


Рисунок 11 – Модель сервиса

Ниже на рисунке 12 приведена диаграмма декомпозиции A0.

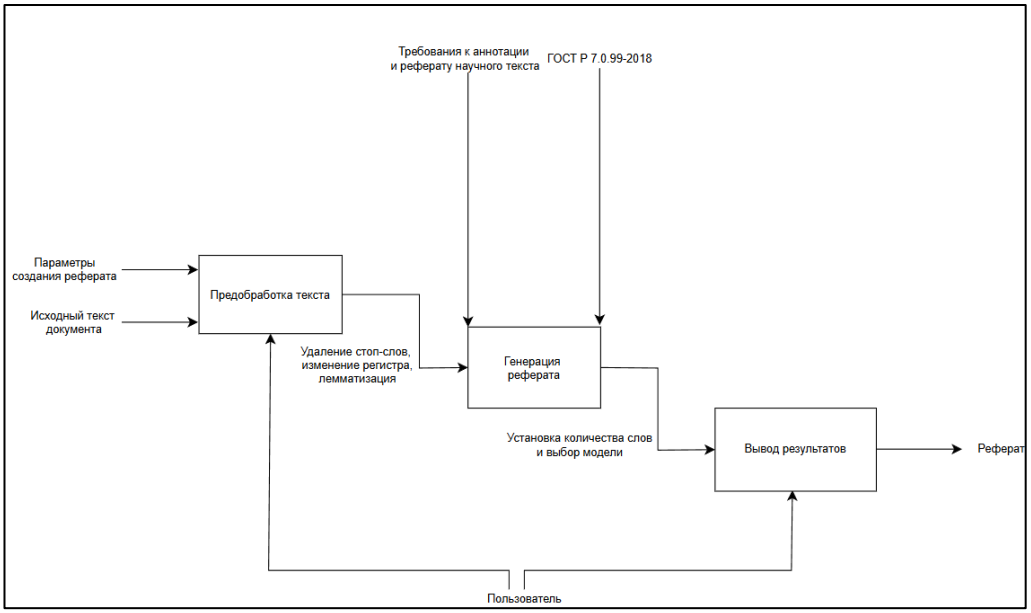


Рисунок 12 – Диаграмма декомпозиции A0

Далее приведем диаграмму прецедентов на рисунке 13.

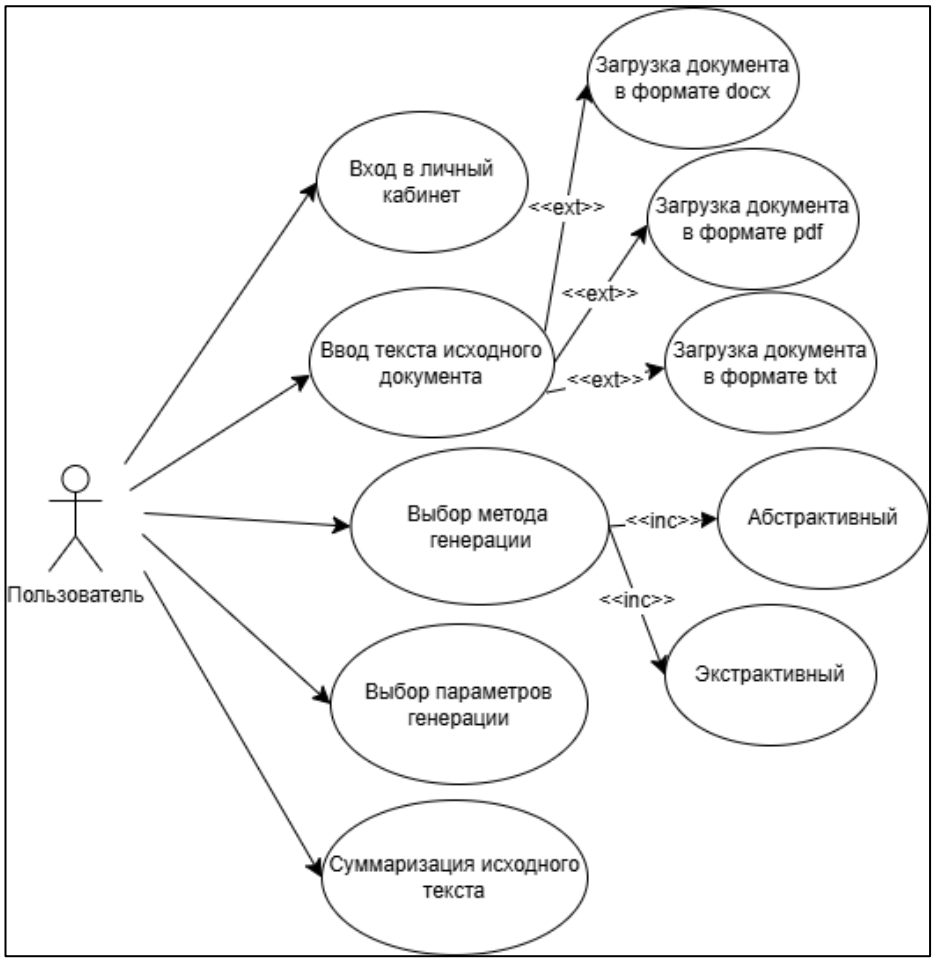


Рисунок 13 – UML-диаграмма прецедентов

Разрабатываемое приложение должно решать следующие задачи:

– ввода текста или загрузки файлов;

– задания параметров генерации (количество слов/символов, выбор модели);

– вывода результата с возможностью копирования;

– генерации реферата;

– регистрации пользователей;

– просмотра предыдущих запросов для авторизованных пользователей.

Подробнее рассмотрим каждую из задач, чтобы точно обозначить требования, предъявляемые к ним. Так, в числе требований для задачи «ввода текста или загрузки файлов» обозначим, что приложение должно предоставлять возможность ввода в текстовое поле текста вручную, загрузку

файлов в различных форматах, извлечение текстового содержимого из предоставленного файла с удалением лишних пробелов и иных символов, также максимальный размер файла должен быть ограничен и составлять не более 10 МБ, а в случае невозможности извлечь текст пользователю необходимо вывести уведомление.

Для задачи «задания параметров генерации» выделим то, что приложение должно предоставлять пользователю выбор между экстрактивными методами и абстрактивными моделями, а также их конкретными видами, также для экстрактивного способа должна быть возможность указать максимальное и минимальное количество предложений в итоговом реферате, а для абстрактивного способа должна быть возможность указать максимальную длину.

Задача «генерации реферата» должна удовлетворять требованию по выполнению суммаризации выбранным методом с учетом заданных параметров длины, показывать пользователю время выполнения запроса и количество предложений в полученном реферате.

Для задачи «вывода результата с возможностью копирования» необходимо выполнить требование по предоставлению пользователю кнопки для копирования реферата. Задача «регистрации пользователей» должна удовлетворять требованию по предоставлении возможности регистрации пользователя с указанием логина и пароля, обеспечению входа зарегистрированных пользователей по логину и паролю, привязыванию каждого запроса по реферированию к текущему зарегистрированному пользователю.

Также, в числе требований для задачи «просмотра истории запросов для авторизованных пользователей обозначим, что приложение должно предоставлять пользователю страницу со списком его запросов, отсортированных по дате, для каждого запроса необходимо отображать его тип метода, дату, начало исходного текста и начало получившегося реферата, а при нажатии на запрос можно было увидеть полную информацию.

Ниже на рисунке 14 приведена диаграмма требований (вариантов использования) UML отражающая основной функционал сервиса.

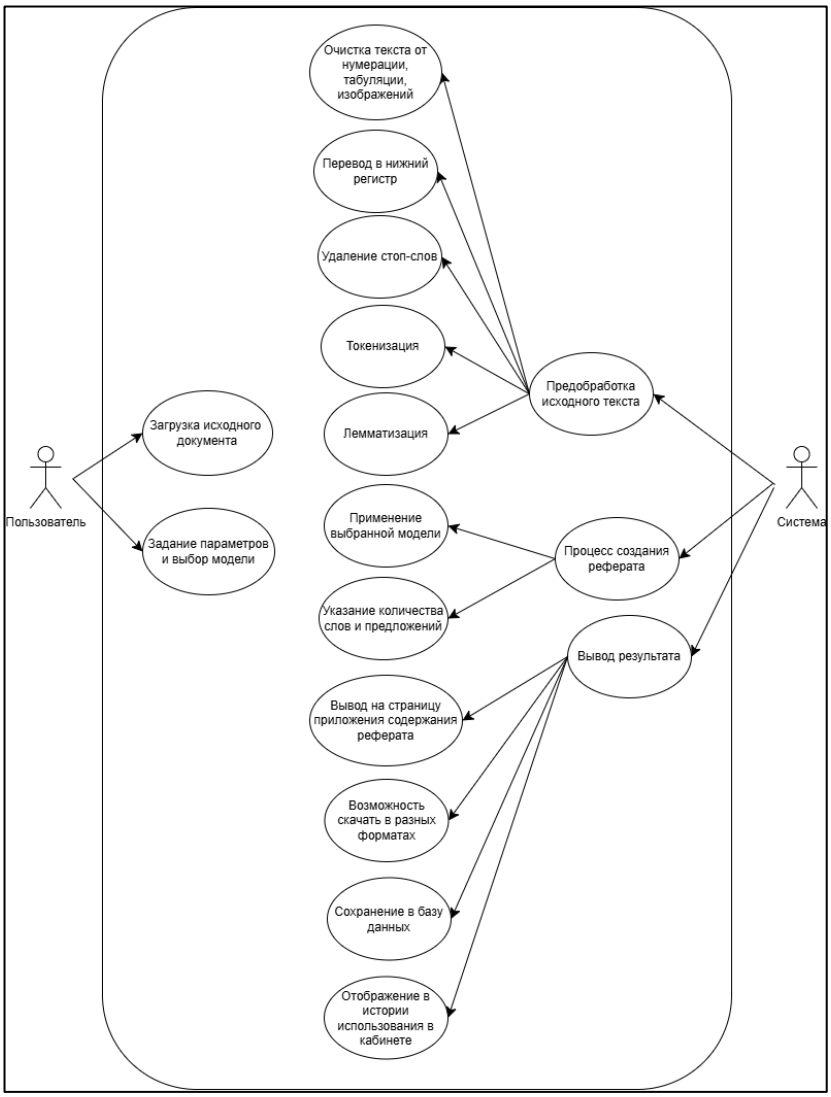


Рисунок 14 – Модель требований в нотации UML

Далее приведем диаграмму состояний (statechart diagram), которая описывает состояние генерации реферата на протяжении всего цикла работы сервиса (рисунок 15).

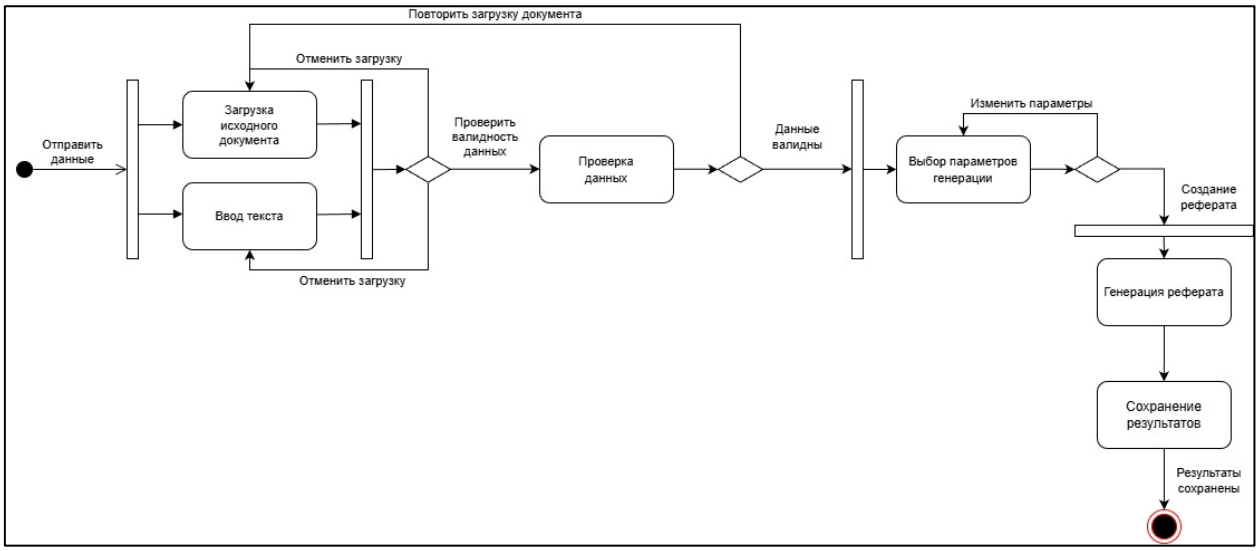


Рисунок 15 – Диаграмма состояний

Ниже приведем диаграмму активности (activity diagram). Это диаграмма, описывающая процесс создания аннотации (рисунок 16).

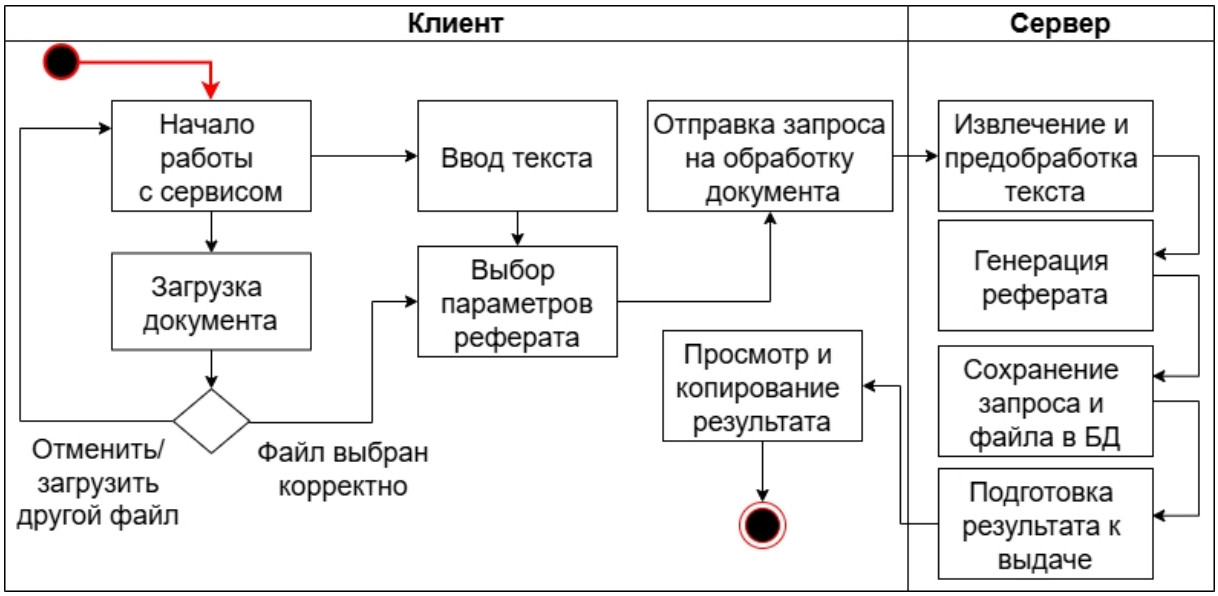


Рисунок 16 – Диаграмма активности

Кроме того, выделим основные нефункциональные требования к разрабатываемому приложению. Так, для удобства пользователя сервис должен обладать приятным и интуитивным интерфейсом, кроме того, сервис должен быть бесплатным и свободно распространяемым, а также иметь открытый исходный код для возможности дальнейшей модификации.

Далее приведена диаграмма последовательностей (sequence diagram), которая описывает жизненный цикл объекта в рамках прецедента (рисунок 17).

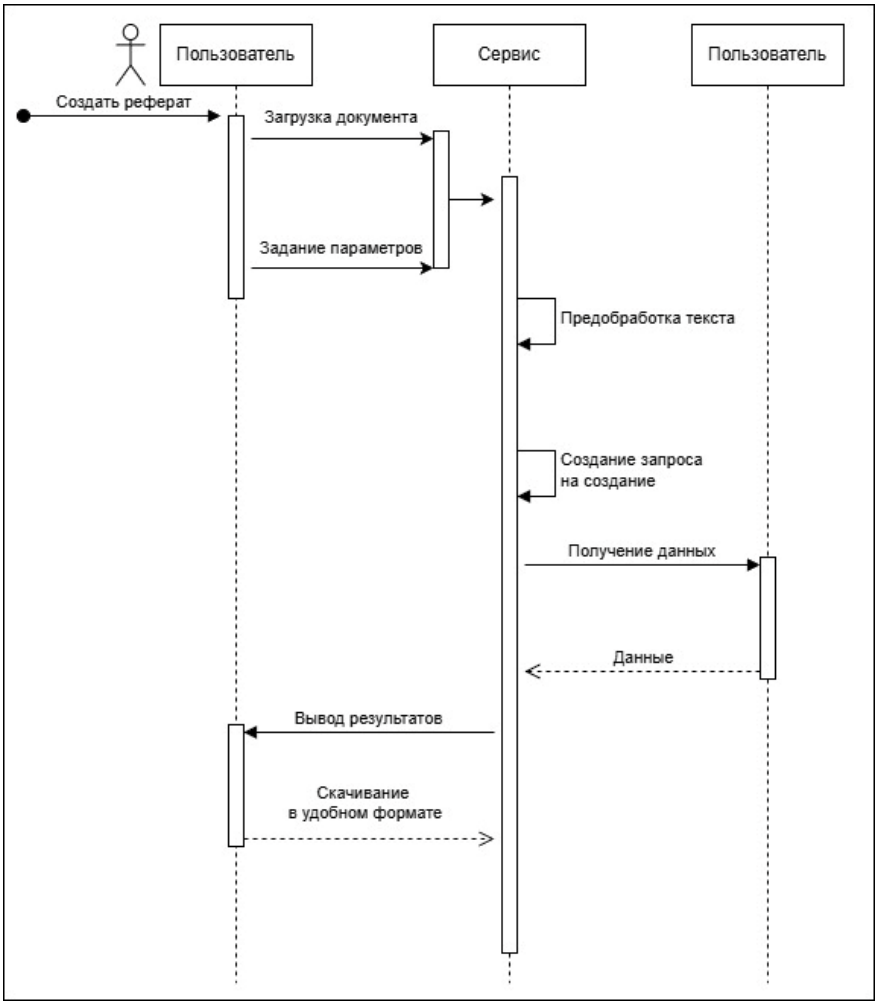


Рисунок 17 – Диаграмма последовательностей

4 Особенности реализации приложения

Для практической реализации и апробации различных методов реферирования разработано веб-приложение на языке Python с использованием фреймворка Django и Django Rest Framework (DRF) для построения клиент-серверного REST API. Приложение предоставляет пользователям функционал регистрации и авторизации, а также возможность выбора метода суммаризации и генерации реферата с последующей возможностью просмотра полного списка предыдущих использований приложения.

4.1 Использованные модели

Для использования различных способов суммаризации используются библиотеки Sumy, Hugging Face Transformers и PyTorch. Доступны как экстрактивные алгоритмы (TextRank, LSA, LexRank), так и абстрактивные модели на основе архитектуры T5 и BART. Для работы абстрактивной суммаризации используются предобученные модели, загрузка которых в оперативную память происходит только при первом обращении к ним. Перечислим конкретные модели ниже.

Модель rut5-base-sum-gazeta представляет собой дообученную на русскоязычном датасетей новостных статей с сайта Gazeta.ru. Модель хорошо работает на новостных текстах и показывает высокие метрики качества. Однако, она может работать хуже при смене области домена, так она не будет подходить для, например, научных статей, юридических и медицинских документов и т.д.

Модель rut5-base-absum является более легкой моделью той же архитектуры и задачи. Она подходит для более широкого применения, так как в качестве обучающих корпусов были использованы пары «исходный текст – резюме», собранные с различных новостных сайтов, таких как BBC и CNN, а

также датасета WikiLingua, составленного из руководств с сайта WikiHow на различные тематики. Так как модель имеет меньше параметров она показывает качество рефератов ниже, чем предыдущая модель.

Модель mbart-ru-sum-gazeta основана на архитектуре mBART, то есть многоязычного двунаправленного и авторегрессивного трансформера. Модель обучена на таких же наборах, как и первая приведенная, и из-за того, что задача изначально состояла в создании исходного текста из замаскированного и использовала больше параметров, она тоже хорошо подходит для задачи суммаризации.

Общая схема доступных для использования моделей и наборов, на которых они обучались, показана на рисунке 18.

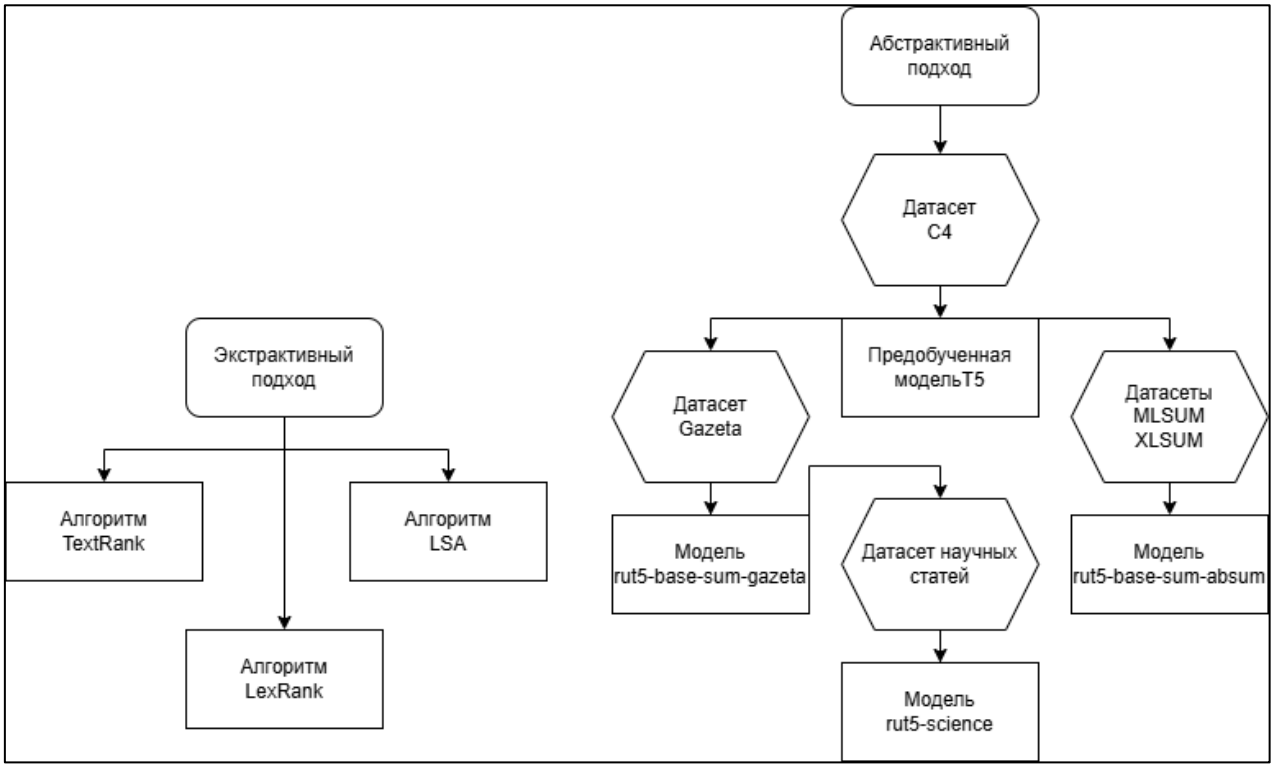


Рисунок 18 – Использованные подходы

4.2 Пример работы приложения

Пользователь может ввести текст вручную либо загрузить документ в форматах PDF, DOCX или TXT с использованием библиотек pypdf и python-

docx для извлечения текстового содержимого. Выполняется валидация файлов на предмет правильного размера файла, т.е. до 10 МБ, и его расширения. Для файлов формата TXT проверяется несколько возможных кодировок. Для файлов формата PDF текст извлекается с отдельных страниц, а объединяется, для файлов формата DOCX аналогично, но с параграфами. Результат работы может быть скопирован в буфер обмена, а все запросы с указанием исходного текста, полученного реферата и загруженных файлов сохраняются в личном кабинете пользователя для последующего просмотра в порядке от новых к старым. Для каждого запроса показываются такие детали, как тип использованного подхода, параметр длины, время обработки и дата запроса, а также первые предложения исходного и финального текста. Основные функциональные интерфейсы приведены в таблице 1.

Таблица 1 – Основные эндпоинты API

Метод	*Путь*	*Описание*
POST	/api/requests/summarize/	Создание запроса на реферирование
GET	/api/requests/	Получение списка запросов пользователя
POST	/api/files/upload/	Загрузка файла
GET	/api/files/	Получение списка загруженных файлов

Все эндпоинты использую формат JSON для передачи данных и строятся на стандартных HTTP-методах. Для достижения всех эндпоинтов, за исключением страниц аутенфикации, требуется наличие Basic Auth-токена. Это обеспечивает доступ пользователя только к своей истории запросов и загруженных файлов. Для предотвращения подделки POST-запросы требуют передачи CSRF-токена, который автоматически подставляется в формы Django и в заголовки запросов. Также, при возникновении ошибок пользователю возвращается всплывающее сообщение без технических подробностей, а сама ошибка сохраняется в логах.

В проекте использована архитектурная парадигма MVT (Model-View-Template), являющаяся вариацией классической MVC. В рамках данной архитектуры определены модели данных: SummaryRequest для хранения текста запроса, результата и типа суммаризации, а также UploadedFile для хранения метаинформации о загруженных файлах.

Бизнес-логика обработки запросов на реферирование инкапсулирована в классе SummaryRequestViewSet. Класс имеет классический CRUD функционал, а также метод summarize. Так, при получении запроса на эндпоинт метод обработает его в несколько шагов. Во-первых, будет совершена проверка наличия текста и корректности заданных параметров. Во-вторых, определен тип метода и вызов наследника соответствующего класса ExtractiveSummarizer или AbstractiveSummarizer. По итогу результат будет сохранен в базу данных и возвращен JSON-ответ с рефератом, его длиной и временем генерации. Также в классе FileUploadViewSet реализованы методы upload для обработки загружаемых файлов и summarize для реферирования извлеченного из файла текста.

Для создания динамичного и реактивного пользовательского интерфейса на клиентской стороне применяются технологии HTMX и Alpine.js, что позволяет обновлять отдельные части страницы без полной перезагрузки.

В начале работы с приложением пользователь попадает на страницу входа, где ему необходимо ввести логин и пароль. Если таковых нет, то он сможет зарегистрироваться по ссылке ниже. В таком случае нужно будет придумать логин и пароль, состоящий не менее чем из шести символов, и затем подтвердить пароль. Тогда пользователь будет сохранен в базе данных и его переадресует на страницу входа.

Пример работы приложения представлен ниже на рисунке 19.

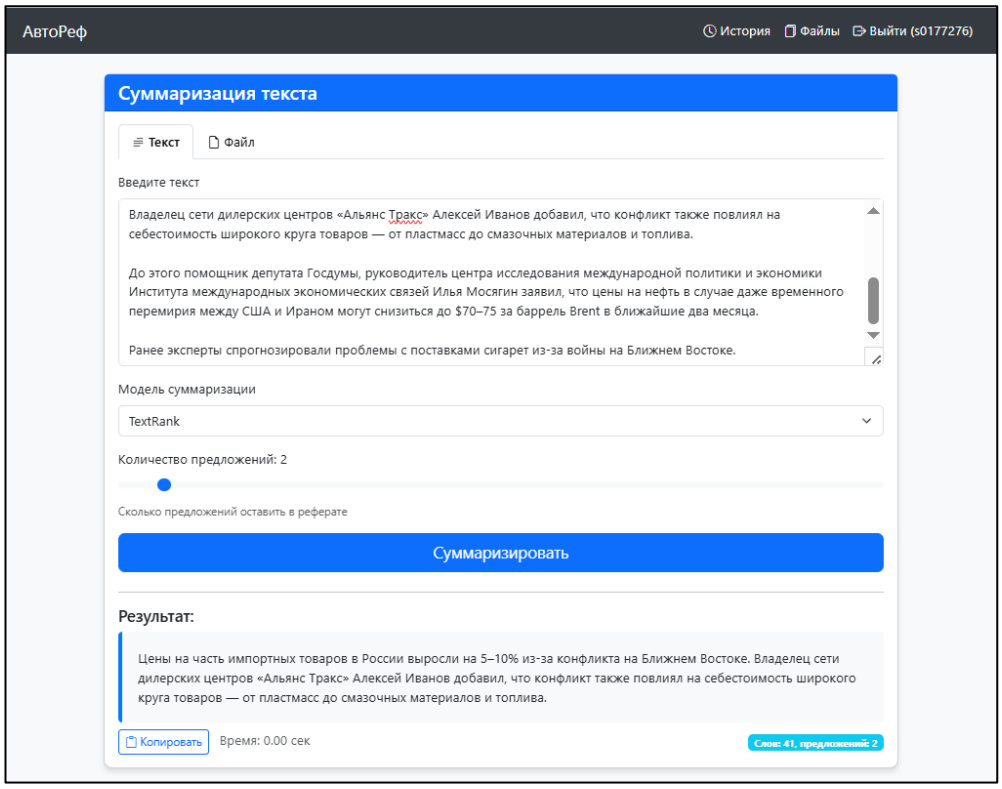


Рисунок 19 – Суммаризация статьи

Также в разделе история можно просмотреть полностью предыдущие запросы пользователя. Пример раздела представлен на рисунке 20.

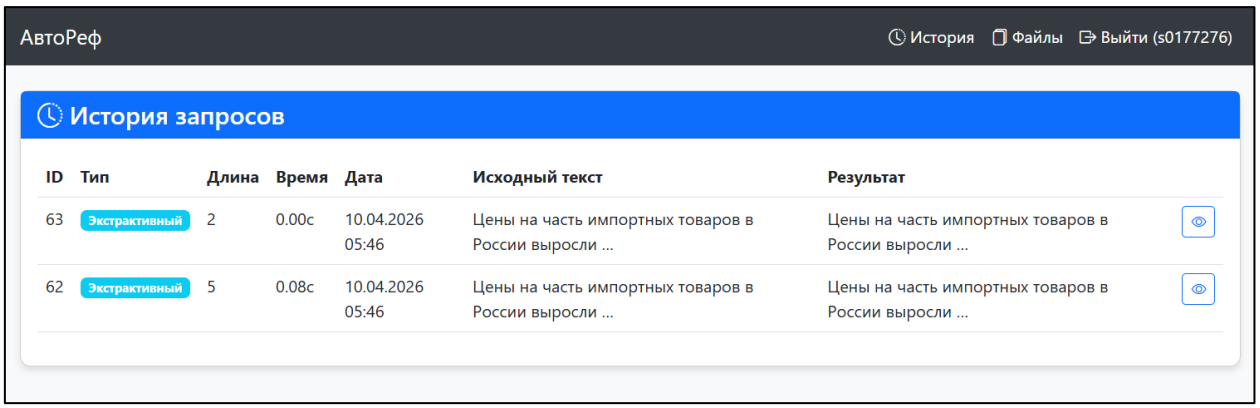


Рисунок 20 – Предыдущие запросы пользователя

5 Дообучение модели rut5 для решения задачи реферирования научных статей

Обучение модели для выполнения задачи реферирования научных текстов было выполнено в два этапа: начальное для обучения модели задачи суммаризации на датасете Gazeta, состоящем из 60 тысяч пар новостной статьи и ее краткой аннотации. Так, длина входной последовательности токенов была установлена 600, а выходной - 200. Была выбрана модель T5 (Text-to-Text Transfer Transformer), отличающаяся унифицированным подходом к решению гетерогенных задач через конверсию в формат «текст-на-входе – текст-на-выходе». Модель эффективно усваивает языковые закономерности, так как обучена на реконструкции пропущенных фрагментов. Была выбрана специализированная версия модели ruT5 для русского языка.

Применение такой модели ограничено доменом датасета, используемого для обучения, поэтому был предпринят второй этап, который является адаптацией модели под научную область. Так, был собран датасет пар текст научной статьи и ее аннотация. В него входит 912 текстов разной предметной области: компьютерные науки, медицина, юриспруденция и т.д.

Для второго этапа была снижена скорость обучения, так как иначе обучение может быть нестабильным. В связи с этим было увеличено число эпох по сравнению с первом этапом до 8. Также для исключения катастрофического забывания умения к реферированию в датасет второго этапа было добавлено 200 примеров из датасета Gazeta.

Обучение проводилось с помощью библиотеки hugging face transforemers с оптимизатором AdamW и сохранением в качестве итоговой модели лучшего из чекпоинтов по метрике Rouge-L на валидационной выборке.

Далее с помощью метрик Rouge-1, Rouge-2, Rouge-L, METEOR, BERTScore, chrF было проведено сравнение экстрактивных методов – TextRank, LexRank, LSA и абстрактивных моделей – rut5-base-absum, rut5-

base-sum-gazeta и дообученной модели на научных текстах. Результаты представлены ниже в таблице 2.

Таблица 2 – Сравнение результатов работы

Модель	ROUGE-1	ROUGE-2	ROUGE-L	chrF	METEOR	BERTScore
ruT5-base-absum	9.95	2.96	9.95	19.92	10.4	**88.34**
ruT5-base-sum-gazeta	**14.68**	**5.11**	**14.21**	35.54	20.34	87.78
ruT5-science	13.97	3.68	13.73	35.17	17.08	86.24
TextRank	9.96	2.87	9.37	**35.97**	**23.36**	87.17
LexRank	9.59	3.32	9.54	33.38	19.63	87.46
LSA	9.34	3.72	9.31	34.2	21.18	86.54

Кроме сравнения метрик также была проведена экспертная оценка, по итогам были найдены различия. Экспертная оценка включает такие критерии, как длина реферата, использование профессиональных терминов и наличие в реферате основных частей статьи. Во-первых, дообученная модель генерировала более длинные рефераты, что связано с различием данных обучения. Во-вторых, дообученная модель чаще использовала термины из текста, делая текст более научным, но вместе с тем используя длинные и сложные по структуре предложения. Также дообученная модели чаще галлюцинирует и заканчивают слова неправильными буквами. Примеры сгенерированных рефератов и результаты их сравнения представлены в Приложении А.

6 Описание GPT-активностей

В условиях активного использования генеративных нейронных сетей (ГНС) возникает необходимость фиксирования процесса взаимодействия с ними для повышения понимания степени успешности различных практик и последующего их широкого использования. Так, в процессе работы был задокументирован процесс написания промптов и взаимодействия с ИИ-моделями (DeepSeek, ChatGPT, Grok, Claude), проведена оценка результата и произведен фактчекинг. Результат анализа представлен в таблице 3.

Таблица 3 – Матрица промптов

№	Категория задачи	Пример промпта	Цель	Результат и фактчекинг
1	Поиск литературы	Ты — эксперт по обработке естественного языка, специализирующийся в сфере автореферирования и суммаризации. Твоя задача — привести 5 научных статей не ранее 2021 года на русском языке о современных исследованиях в твоей области	Выявить базу для теоретической части	Все статьи оказались реально существующими и релевантными, отражающими современные тенденции в сфере.
2	Поиск литературы	Проведи аналитический обзор первой из предложенных тобой статьи	Выявить базу для теоретической части	В большей части обзоров ГНС действительно правдиво обозревала статью, но были и галлюцинации, такие как изменение имени автора на случайные, изменение величины метрик моделей

Продолжение таблицы 3

3	Поиск литературы	Проведи поиск актуальных научных статей на русском языке про метрики оценки качества сгенерированного текста	Выявить базу для теоретической части	ГНС предлагал статьи, действительно подходящие общей тематики работы, при этом специфичный запрос, по его мнению, он тоже выполнял, но в реальных статьях по ссылке часто не было никакой глубокой информации о метриках и их вычислении, кроме самого их использования.
4	Генерация кода	Напиши код на Python для загрузки и обработки датасета IIS summarization dataset https://github.com/iis-research-team/summarization-dataset	Автоматизировать сборку обучающего корпуса	Код был сгенерирован правильно. Для проверки было выведено общее количество пар и несколько случайных пар, чтобы удостовериться в их непустоте.
5	Генерация кода	Реализуй класс ExtractiveSummarizer с помощью библиотеки Sumy с тремя методами TextRank, LexRank, LSA	Создание модуля экстрактивной суммаризации	Код был сгенерирован без ошибок, но оказалось, что не было настроено исключение стоп-слов. При сообщении этой ошибки вместо использования какой-либо библиотеки было предложено создать свой список в формате txt.
6	Генерация кода	Реализуй метод summarize, принимающий входной текст, модель, длину реферата для модели SummaryRequest	Реализация эндпоинта для реферирования	Код сгенерирован корректно, но поле итогового реферата было указано только для чтения, поэтому результат не сохранялся.
7	Коррекция ошибок	Объясни причину и исправь предупреждение TypeError: Seq2SeqTrainer.__init__() got an unexpected keyword argument 'tokenizer'	Исправление ошибки, связанной с версией transformers	Ошибка заключалась в том, что в актуальной версии библиотеки tokenizer используется через объект DataCollatorSeq2Seq, а не передается через Trainer. Новый код был сгенерирован корректно и Trainer инициализирован.

Продолжение таблицы 3

8	Коррекция ошибок	Обучение завершилось без ошибок, но best_model_checkpoint равен None и best_metric не определено, объясни и исправь ошибку	Выявить причину отсутствия чекпоинта	ГНС объяснила, что число шагов, предпринятых при обучении, оказалось меньше eval_steps и фиксирования результата необходимо сменить параметры оценки со steps на epoch. После этого изменения чекпоинт стал сохраняться корректно.
9	Подбор гиперпараметров	При обучении модели ValidationLoss=0, TrainingLoss=0 и Rouge-метрики не меняются, объясни причину и исправь	Скорректировать процесс обучения модели	ГНС предположила, что такая нестабильность может быть связана с включенным параметром fp16. Так, после его выключения и проверки на одном батче loss стал отличным от нуля и eval_loss стал уменьшаться.
10	Сравнение метрик	Построй таблицу сравнения методов TextRank, LSA, LexRank и моделей rut5-base-absum, rut5-base-sum-gazeta и rut5-science на метриках Rouge-1, Rouge-2, Rouge-L, chrF, METEOR, BERTScore	Наглядно оценить различные методы и модели	Код был сгенерирован без ошибок, наибольшее значение для конкретной метрики было выделено.

7 Рефлексивный анализ

В ходе выполнения работы применение ГНС в большинстве случаев дало положительный результат. Так, для задачи начального обзора источников в области автоматической суммаризации текстов применение ИИ является оправданным, так как все приведенные им статьи были актуальными и были изданы не ранее установленного года, однако, вероятно, в некоторых случаях ИИ не смог достичь текста оригинальной статьи и придумывал правдоподобное содержимое, чаще всего он ошибался в количественных характеристиках и оценках, так как он не мог извлечь их из графиков и таблиц, а также иногда придумывал вероятные имена авторов вместо приведения исходных.

Для задачи реализации программного кода ИИ также является допустимым вариантом, поскольку чаще он генерирует корректный код, но его реализация периодически не соответствует актуальной версии библиотеки. Так, нередкими являлись конфликты, связанные с устаревшими версиями библиотек и подходами к написанию кода. В то же время и простые ошибки выполнения кода нередко имели место.

Таким образом, ИИ является полезным инструментом, экономящим время, для большого круга задач. Вместе с этим результаты генерации должны подвергаться критическому анализу, следует перепроверять неочевидные утверждения. Использование ГНС оправдано в задачах, требующих большого количества простых и понятных ему шагов, однако в сложных решениях следует перепроверять его мысли в исходных источниках или других ГНС. Следует также, как можно более точно формулировать свои запросы для получения более предсказуемых результатов и уточнять детали, если с первого раза результат не получился.

ЗАКЛЮЧЕНИЕ

Цель выпускной квалификационной работы – проектирование и реализация приложения автоматизированной генерации реферативных текстов – достигнута.

В теоретической части выпускной квалификационной работы описаны современные методы и модели для решения задачи суммаризации.

Выполнен реферативный обзор сервисов, реализующих генерацию реферативных научных текстов. Спроектировано приложение для автоматизированной генерации реферативных текстов.

В работе предложен проект и реализация приложения автоматизированной генерации рефератов. Основной функцией сервиса является генерация кратких аннотаций на основе исходных текстов с возможностью выбора метода и настройки параметров длины выходного текста. Также был предложен и реализован двухэтапный подход к обучению модели ruT5 для адаптации к реферированию русскоязычных научных статей.

При программной реализации функционала использовался фреймворк языка Python Django. Сервис решает такие задачи, как выполнение абстрактивной суммаризации текста с помощью нейросетевых моделей архитектур BART и T5 и экстрактивной суммаризации с использованием алгоритмов TextRank, LSA и LexRank, извлечение текстового содержимого из файлов разных форматов и отображение прошлых запросов в истории для авторизованных пользователей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Сорокина, С.Г. Интеллектуальная обработка текстовой информации: обзор автоматизированных методов суммаризации/ С.Г. Сорокина // Виртуальная коммуникация и социальные сети. – 2024. – №3. – С. 203–222.

2. Медяник, Ю.В. К вопросу о применении методов экстрактивной суммаризации для составления резюме научного текста / Ю.В. Медяник, Л.А. Сабирова // Международный журнал гуманитарных и естественных наук. – 2025. – №8. – С. 268–272.

3. Мельничук, Д.В. Сравнение NLP-моделей на задаче суммаризации академических текстов на русском языке/ Д.В. Мельничук, А.В. Носкина // Компьютерная лингвистика и вычислительные онтологии. – 2024. – №7. – С. 54–59.

4. Дагаев, А.Е. Сравнение автоматического обобщения текстов на русском языке/ А.Е. Дагаев, Д.И. Попов // Программные системы и вычислительные методы– 2024. – №4. – С. 13–22.

5. Ребенок, К.В. Эффективность нейросетевых алгоритмов в автоматическом реферировании и суммаризации текста/ К.В. Ребенок // Вестник Новосибирского государственного университета. Серия: Информационные технологии. – 2024. – №4. – С. 49–61.

6. Эволюция метрик качества машинного перевода – Часть 1 // Хабр: [сайт]. – 2025. – URL: https://habr.com/ru/articles/745642/ (дата обращения: 30.10.2025).

7. Beyond One-Size-Fits-All Summarization: Customizing Summaries for Diverse Users // Arxiv: [сайт]. – 2025. – URL: https://arxiv.org/abs/2503.10675 (дата обращения: 30.10.2025).

8. Эволюция метрик качества машинного перевода – Часть 2 // Хабр: [сайт]. – 2025. – URL: https://habr.com/ru/articles/748496/ (дата обращения: 30.10.2025).

9. ГОСТ Р 7.0.7–2021. Статьи в журналах и сборниках: национальный стандарт Российской Федерации: издание официальное: утвержден и введен в действие приказом Федерального агентства по техническому регулированию и метрологии от 18 августа 2021 г. № 728-ст : взамен ГОСТ Р 7.0.7–2009 : дата введения 2021-10-01 / разработан Российской книжной палатой, филиалом Федерального государственного унитарного предприятия «Информационное телеграфное агентство России (ИТАР-ТАСС)», Ассоциацией научных редакторов и издателей (АНРИ). – Москва : Стандартинформ, 2021. – III, 26 с. – (Система стандартов по информации, библиотечному и издательскому делу).

10. ГОСТ Р 7.0.99-2018 (ИСО 214:1976). Реферат и аннотация. Общие требования: национальный стандарт Российской Федерации: : национальный стандарт Российской Федерации : издание официальное: утвержден и введен в действие приказом Федерального агентства по техническому регулированию и метрологии от 1 августа 2018 г. № 446–ст : введен впервые: дата введения 2019-01-01 / подготовлен Федеральным государственным бюджетным учреждением науки «Всероссийский институт научной и технической информации Российской академии наук (ВИНИТИ РАН)», Федеральным государственным бюджетным учреждением «Российская государственная библиотека», Федеральным государственным бюджетным учреждением «Государственная публичная научно-техническая библиотека России». – Москва : Стандартинформ, 2018. – IV, 23 с. – (Система стандартов по информации, библиотечному и издательскому делу).

11. Постановка задачи автоматического реферирования и методы без учителя // Хабр: [сайт]. – 2025. – URL: https://habr.com/ru/articles/595517/ (дата обращения: 27.10.2025).

12. Извлекающие методы автоматического реферирования // Хабр: [сайт]. – 2025. – URL: https://habr.com/ru/articles/595597/ (дата обращения: 30.10.2025).

13. Секреты генерирующего реферирования текстов // Хабр: [сайт]. – 2025. – URL: https://habr.com/ru/articles/596481/ (дата обращения: 30.10.2025).

14. Sequence-to-Sequence (seq2seq) and Attention // Lena Voita: [сайт]. – 2025. – URL: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html (дата обращения: 29.10.2025).

15. Scholarcy // Scholarcy Limited: [сайт]. – 2025. – URL: https://www.scholarcy.com/ (дата обращения: 28.11.2025).

16. QuillBot // Learneo: [сайт]. – 2025. – URL: https://quillbot.com/ (дата обращения: 28.11.2025).

17. SciSummary // SciSummary: [сайт]. – 2025. – URL: https://scisummary.com/ (дата обращения: 29.11.2025).

ПРИЛОЖЕНИЕ А

Сравнение сгенерированных рефератов научных статей

Оценку и сравнение рефератов будем проводить по экспертным критериям. В их числе: длина реферата, использование профессиональных терминов и наличие в реферате основных частей статьи.

Так, сначала можно заметить, что дообученная модель чаще предлагает резюме большей длины. Для первого примера: 28 слов и 57 слов, 2 предложения в обоих случаях, 1 придаточное предложение в обоих случаях, использован деепричастный оборот во втором случае. Для второго примера имеем: 19 слов и 21 слово, в обоих случаях одно предложение, в первом случае 1 причастный оборот, во втором 1 придаточное предложение. Для третьего примера: 36 слов и 57 слов, 2 предложения в обоих случаях, 1 придаточное предложение в первом случае, 2 причастных и 1 деепричастный оборота во втором случае.

Далее рассмотрим тексты на вопрос использования терминов для составления резюме. Так, в первом тексте во втором случае употребляются термины «абонементального» и «эксцессов». Во втором использованы вещества и метод, которые участвовали в исследовании. В третьем случае оба текста имеют довольно популярный язык рассказа.

Кроме того, рассмотрим тексты на наличие основных элементов статьи, то есть методов и выводов работы. В первом тексте в обоих случаях представлены выводы, во втором случае указаны результаты («выяснилось, что») и методы («при использовании»). Во втором тексте в обоих примерах указаны выводы(«продемонстрировали»), а также методы, использованные в работе. В третьем тексте также сказано о выводах («выяснилось» и «предлагаем») в обоих случаях, при этом методы более точно отражены во втором случае («основанными на»).

Таблица А.1 – Примеры сгенерированных рефератов научных статей

Модель	ruT5-base-sum-gazeta	ruT5-science
Пример первого реферата	Групповая преступная деятельность представляет собой любое взаимодействие нескольких лиц, содержанием которой становится участие одних людей в совершении преступления со стороны остальных. Такое мнение выдвинул Пленум Верховного Суда (РСФСР).	Выяснилось, что совершение группового преступления представляет собой любое взаимодействие некоторых физических лиц — содержанием которой в этой же связи выступают все соучастники из числа субъектов противостояния или недостижения уровня уголовного заключения состава преступления. Ранее такое высказывание сделано при использовании определения «групповых эксцессов», объясняя эту ситуацию как способ выполнения главной стороны расставания по отношению к конкретному составу преступления.
Пример второго реферата	Ученые продемонстрировали возможности получения целлюлозы из растительного сырья в присутствии экологически безопасных катализаторов, позволяющих избежать негативного влияния окружающей среде.	Исследователи продемонстрировали возможность получения целлюлозы с использованием способа каталитической пероксидной делигнификации древесины пихты, из которой удалена гемицеллюкарбоная или микрокристаллическая целлюлоза (NH4)6Mo7O24.
Пример третьего реферата	Выяснилось, что выявление этих слухов позволяет развенчать этот процесс и предотвратить причинение вреда на общественное мнение. Таким образом мы ввели новую категорию функций по распознаванию всех возможных проблем из-за короткого текста или большего объема потоков информации.	В социальных сетях появилось множество функций, позволяющих развенчать распространение слухов или преднамеренную информацию об уголовном преследовании беженцев в 2016 году. Мы предлагаем использовать новую категории возможностей: функциями, основанными на новизне, компенсируют отсутствие повторяющейся информации из-за коротких текстовых вариаций с большим объемом потоков данных – новостным статьям со ссылкой к дополнительным СМИ – сообщениям пользовательской аудитории, не заглядываясь в будущее.